



Assignment 4

Aim:- Implement Search Tree which is suitable for E-commerce platform - a product search system in an E-commerce database.

Algorithm - ① Define structure

```
struct product {  
    int id;  
    char name[50];  
    float price;  
};
```

```
struct Node {  
    product product;  
    struct Node *left;  
    struct Node *right;  
    int height;  
};
```

② write helper function

- define `getHeight(node)` to get the height of tree.
- define `getBalance(node)` as `height(left) - height(right)`
- Define `rightRotate(y)` and `leftRotate(x)` to restructure unbalanced nodes and update their heights.

③ Insertion.

④ If the tree or subtree is empty allocate and return a new Node with height 1;

⑤ Compare string with the current nodes key. using `strcmp()` and perform Appropriate operation recursively.



iii) Update the height of ancestor node
 $height = 1 + \max(height(left), height(right))$

④ Calculate the Balance factor of the current node.

⑤ If the balance factor is unbalanced (>1 or <-1)
Apply Rotation.

- LL case (balance >1 & $id < left \rightarrow id$)
call rightRotate(node)
- RR case (balance <-1 and $id > right \rightarrow id$)
call leftRotate(node);
- LR case (balance >1 & $id < left \rightarrow id$)
call leftRotate(node $\rightarrow$ left) then rightRotate(node)
- RL case (balance <-1 & $id < right \rightarrow id$)
call rightRotate(node $\rightarrow$ right) then leftRotate(node)

4. Print catalog

- print the Tree via Inorder traversal.

Conclusion:- The E-commerce product data structure was successfully implemented in C using AVL Tree. Using a static dataset of products.

```

#include <stdio.h>

#include <stdlib.h>

#include <string.h>


// 1. Define the E-Commerce Product Data Structure
typedef struct Product {
    int id;           // Key used for indexing/searching
    char name[50];
    float price;
} Product;


// 2. Define the AVL Tree Node Structure
typedef struct Node {
    Product product;
    struct Node *left;
    struct Node *right;
    int height;
} Node;


// Utility function to get the height of a node
int getHeight(Node *n) {
    if (n == NULL) return 0;
    return n->height;
}


// Utility function to get the maximum of two integers
int max(int a, int b) {
    return (a > b) ? a : b;
}

```

// Create a new AVL Tree Node

```
Node* createNode(Product p) {  
    Node* node = (Node*)malloc(sizeof(Node));  
    node->product = p;  
    node->left = NULL;  
    node->right = NULL;  
    node->height = 1; // New node is initially added at leaf level  
    return node;  
}
```

// Get Balance factor of node N

```
int getBalance(Node *n) {  
    if (n == NULL) return 0;  
    return getHeight(n->left) - getHeight(n->right);  
}
```

// Right rotate subtree rooted with y

```
Node* rightRotate(Node *y) {
```

```
    Node *x = y->left;
```

```
    Node *T2 = x->right;
```

```
    // Perform rotation
```

```
    x->right = y;
```

```
    y->left = T2;
```

```
    // Update heights
```

```
    y->height = max(getHeight(y->left), getHeight(y->right)) + 1;
```

```
    x->height = max(getHeight(x->left), getHeight(x->right)) + 1;
```

```

    return x; // New root
}

// Left rotate subtree rooted with x
Node* leftRotate(Node *x) {
    Node *y = x->right;
    Node *T2 = y->left;

    // Perform rotation
    y->left = x;
    x->right = T2;

    // Update heights
    x->height = max(getHeight(x->left), getHeight(x->right)) + 1;
    y->height = max(getHeight(y->left), getHeight(y->right)) + 1;

    return y; // New root
}

// Insert a Product into the AVL Tree
Node* insert(Node* node, Product p) {
    // 1. Perform standard BST insertion
    if (node == NULL) return createNode(p);

    if (p.id < node->product.id)
        node->left = insert(node->left, p);
    else if (p.id > node->product.id)
        node->right = insert(node->right, p);
}

```



```
else // Duplicate Product IDs are not allowed
    return node;

// 2. Update height of ancestor node
node->height = 1 + max(getHeight(node->left), getHeight(node->right));

// 3. Get the balance factor to check if it became unbalanced
int balance = getBalance(node);

// Left Left Case
if (balance > 1 && p.id < node->left->product.id)
    return rightRotate(node);

// Right Right Case
if (balance < -1 && p.id > node->right->product.id)
    return leftRotate(node);

// Left Right Case
if (balance > 1 && p.id > node->left->product.id) {
    node->left = leftRotate(node->left);
    return rightRotate(node);
}

// Right Left Case
if (balance < -1 && p.id < node->right->product.id) {
    node->right = rightRotate(node->right);
    return leftRotate(node);
}
```

```

    return node;
}

// Search for a product by ID in O(log n)
Node* search(Node* root, int id) {
    if (root == NULL || root->product.id == id)
        return root;

    if (id < root->product.id)
        return search(root->left, id);

    return search(root->right, id);
}

```

```

// In-Order Traversal (Prints entire catalog sorted by Product ID)
void printCatalog(Node* root) {
    if (root != NULL) {
        printCatalog(root->left);
        printf("%-10d %-20s $%.2f\n", root->product.id, root->product.name, root->product.price);
        printCatalog(root->right);
    }
}

```

```

// Free memory allocation
void freeTree(Node* root) {
    if (root != NULL) {
        freeTree(root->left);
        freeTree(root->right);
    }
}

```

```
        free(root);  
    }  
}
```

```
int main() {  
    Node *root = NULL;  
  
    // Sample product dataset  
    Product p1 = {105, "Wireless Mouse", 25.99};  
    Product p2 = {101, "Mechanical Keyboard", 89.99};  
    Product p3 = {110, "Gaming Monitor", 299.50};  
    Product p4 = {102, "USB-C Cable", 12.00};  
  
    // Insert products into the AVL Tree  
    root = insert(root, p1);  
    root = insert(root, p2);  
    root = insert(root, p3);  
    root = insert(root, p4);  
  
    // 1. Display Sorted Catalog (In-Order Traversal)  
    printf("--- E-Commerce Catalog (Sorted by ID) ---\n");  
    printf("%-10s %-20s %-10s\n", "ID", "Name", "Price");  
    printf("-----\n");  
    printCatalog(root);  
  
    // 2. Search for a Product  
    int searchID = 102;  
    printf("\nSearching for Product ID %d...\n", searchID);  
    Node *result = search(root, searchID);
```



```
if (result != NULL) {  
    printf("Found: %s - $%.2f\n", result->product.name, result->product.price);  
} else {  
    printf("Product not found.\n");  
}  
  
// Cleanup  
freeTree(root);  
return 0;  
}
```